Problem Solving by Search

Uninformed (Blind) Search: BFS, DFS, DLS, IDDFS and UCS

1. Introduction

Whenever a program has to find a route, solve a puzzle or plan a sequence of moves, it cannot just know the answer. It has to try things out in some organised way and stop when it reaches what it wants. In Artificial Intelligence this idea is called **problem solving by search**. The program starts from a given situation, looks at the moves that are possible, and keeps building a bigger and bigger picture of where those moves can lead until it finds a goal.

This assignment covers the first family of search methods that students usually meet: the **uninformed** or **blind** strategies. They are called blind because the only thing they know is the problem definition itself. They have no hint such as 'this state looks closer to the goal'. The five algorithms discussed here are Breadth-First Search (BFS), Depth-First Search (DFS), Depth-Limited Search (DLS), Iterative Deepening Depth-First Search (IDDFS) and Uniform-Cost Search (UCS). Informed methods such as greedy best-first search and A* use extra knowledge and are outside the scope of this work.

1.1 How a search problem is described

A search problem is normally written down with five parts:

- **Initial state** - where the agent begins.
- **Actions** - what the agent is allowed to do in a given state.
- **Transition model** - which state you land in after doing an action.
- **Goal test** - a check that tells us whether a state is a goal.
- **Path cost** - a number attached to a path, usually the sum of the step costs.

Exploring the problem produces a **search tree**. The root is the initial state, every branch is an action and every node is a state reached by a particular path. The nodes that have been generated but not yet expanded form the **frontier**. The only real difference between the algorithms below is which node they pick from the frontier next.

1.2 Symbols and the four yardsticks

To compare the algorithms fairly, the following symbols are used throughout:

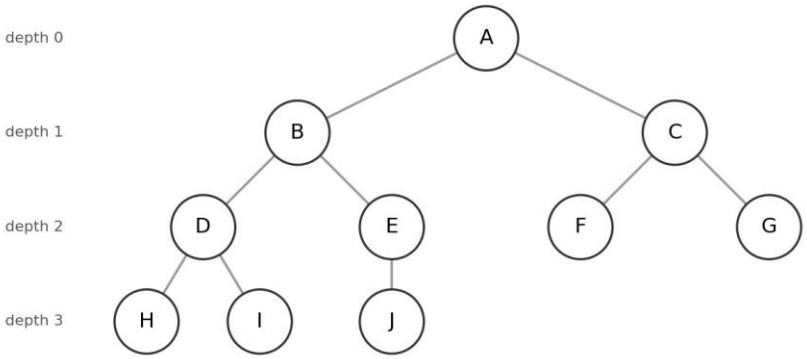
- b = branching factor (maximum number of children of a node)
- d = depth of the shallowest goal
- m = maximum depth of the search tree (can be infinite)
- l = the depth limit used by DLS
- C^* = cost of the cheapest solution, and *epsilon* = the smallest step cost

Each algorithm is judged on four points: **completeness** (will it find a solution if one exists?), **optimality** (is the solution it finds the cheapest?), **time complexity** (how many nodes may it generate?) and **space complexity** (how many nodes must it hold in memory?).

1.3 The example used in this assignment

To keep things easy to follow, BFS, DFS, DLS and IDDFS are all run on the same small tree, shown in Figure 1. The start node is A and the goal is J. Children are always tried from left to right. I used the convention that the goal test is applied when a node is *taken out* of the frontier (expanded), not when it is first created. Some textbooks test earlier, which can change the node counts slightly but not the final answer. UCS needs edge costs, so it gets its own weighted graph in Section 6.

Figure 1: The example search tree (goal = J)



2. Breadth-First Search (BFS)

Idea. BFS explores the tree level by level. It expands the root, then all the root's children, then all the grandchildren and so on. Nothing at depth k+1 is touched before everything at depth k is finished. Think of dropping a stone into a pond: the ripples spread out evenly in all directions.

Data structure. A FIFO queue. New nodes join at the back and the next node to expand is taken from the front, which is what forces the shallow nodes to be handled first.

```
function BFS(start, goal):     frontier <- queue
containing start     explored <- empty set     while
frontier is not empty:         node <- frontier.dequeue()
if node == goal: return path to node         add node to
explored         for each child of node:             if
child not in explored and not in frontier:
               frontier.enqueue(child)
return failure
```

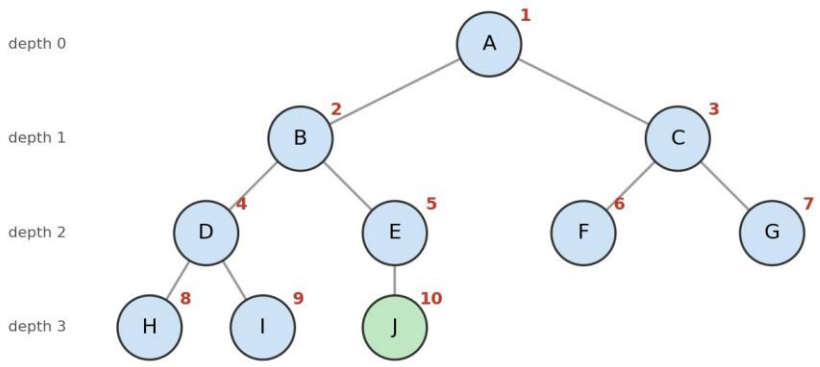
Trace on the example. The queue is written with the front on the left.

Step	Node expanded	Queue after the step
1	A	B, C
2	B	C, D, E
Step	**Node expanded**	**Queue after the step**
3	C	D, E, F, G
4	D	E, F, G, H, I
5	E	F, G, H, I, J

6	F	G, H, I, J
7	G	H, I, J
8	H	I, J
9	I	J
10	J	goal found

BFS expands the nodes in the order **A - B - C - D - E - F - G - H - I - J** and reaches J on the 10th expansion. J is the last node on the bottom level in this left-to-right order, so BFS had to expand every other node of the tree first, which shows its cost when the goal is deep. The path found is A B - E - J.

Figure 2: BFS - nodes numbered in the order they are expanded



Properties.

- **Complete:** yes, as long as the branching factor b is finite.
- **Optimal:** yes when every step costs the same (it finds the shallowest goal). With different step costs it can return a path that has fewer steps but a higher cost.
- **Time:** $O(b^d)$.
- **Space:** $O(b^d)$ - this is the real weakness, because the whole frontier is stored.

Where it is used. Finding the fewest hops in an unweighted network, for example the 'degrees of separation' between two people in a social graph, or the fewest moves in a simple puzzle.

3. Depth-First Search (DFS)

Idea. DFS does the opposite of BFS. It picks one branch and follows it as deep as it can go. Only when it hits a dead end does it back up to the most recent choice point and try the next branch. It is the way most people walk through a maze: keep going until you are stuck, then return to the last junction.

Data structure. A LIFO stack (or plain recursion, which uses the program's call stack). The most recently generated node is the next one to be expanded.

```
function DFS(start, goal):     frontier <-
stack containing start     explored <- empty
set     while frontier is not empty:
```

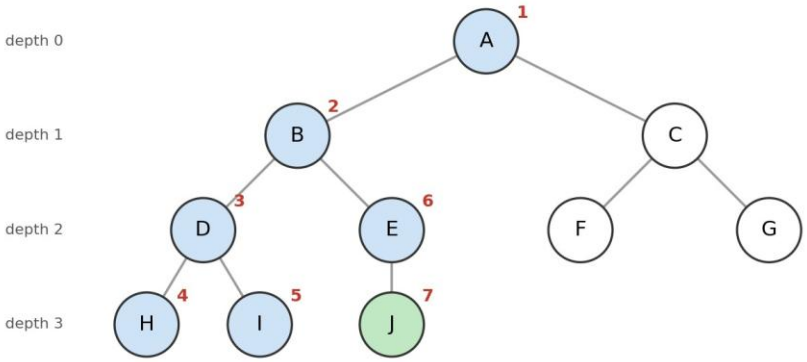
```
node <- frontier.pop()          if node == goal:
return path to node          add node to
explored          for each child of node (right
to left):              if child not in explored:
frontier.push(child)      return failure
```

Children are pushed right to left so that the left-most child is on top of the stack and gets expanded first.

Step	Node expanded	Stack after the step (top on the left)
1	A	B, C
2	B	D, E, C
3	D	H, I, E, C
4	H	I, E, C
5	I	E, C
6	E	J, C
7	J	goal found

The order is **A - B - D - H - I - E - J**: DFS goes down A - B - D - H, backs up to I, backs up again to E and finally reaches J. It needed 7 expansions here compared with 10 for BFS, but that is luck. If the goal had been in the right half of the tree, DFS would have wasted its effort on the whole left half first.

Figure 3: DFS - it dives down the left side first



Properties.

- **Complete:** no in general. In an infinite tree, or in a graph with cycles and no explored-set, it can run forever down one branch. It is complete only in a finite state space when repeated states are checked.
- **Optimal:** no. It returns the first goal it bumps into, which may be deep and expensive.
- **Time:** $O(b^m)$, which is bad if m is much larger than d.
- **Space:** O(b*m) - only the current path and the unexpanded siblings along it are kept. This is the main reason DFS is popular.

Where it is used. Maze solving, detecting cycles in a graph, topological sorting and puzzles where any solution is fine and memory is tight.

4. Depth-Limited Search (DLS)

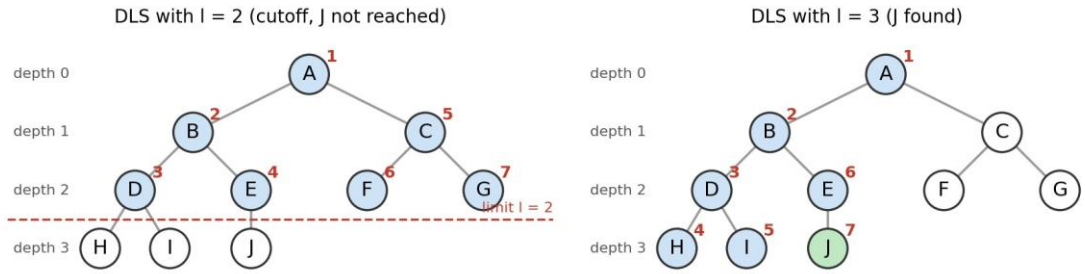
Idea. DLS is DFS with a safety fence. We choose a limit l in advance, and any node at depth l is treated as if it had no children. This stops DFS from disappearing into an infinitely deep branch.

DLS can end in three ways: it finds the **goal**, it returns **failure** (the whole tree was searched and there is no solution), or it returns **cutoff** (the limit stopped the search, so a solution might still exist deeper). Keeping 'cutoff' and 'failure' separate is important, because only 'failure' proves the goal is absent.

```
function DLS(node, goal, limit):
if node == goal: return solution
if limit == 0:   return cutoff
cutoff_occurred <- false     for
each child of node:
        result <- DLS(child, goal, limit - 1)
if result == solution: return solution         if
result == cutoff:   cutoff_occurred <- true     if
cutoff_occurred: return cutoff     else: return
failure
```

Trace on the example. With $l = 2$ the order of visited nodes is **A - B - D - E - C - F - G**. The goal J is at depth 3, so it is just beyond the fence and the answer is **cutoff**. With $l = 3$ the order becomes **A - B - D - H - I - E - J** and J is found.

Figure 4: Depth-limited search with two different limits



Properties.

- **Complete:** no. If l is smaller than d the goal is out of reach.
- **Optimal:** no, even when l is big enough, for the same reason as DFS.
- **Time:** $O(b^l)$.
- **Space:** O(b*l).

The big practical question is how to pick l. Sometimes the problem gives a natural value. For example, if a map has 20 cities, no sensible path needs more than 19 steps, so l = 19 is a safe choice. When nothing like that is known, a wrong guess either misses the goal or wastes time, which is exactly the gap the next algorithm fills.

5. Iterative Deepening Depth-First Search (IDDFS)

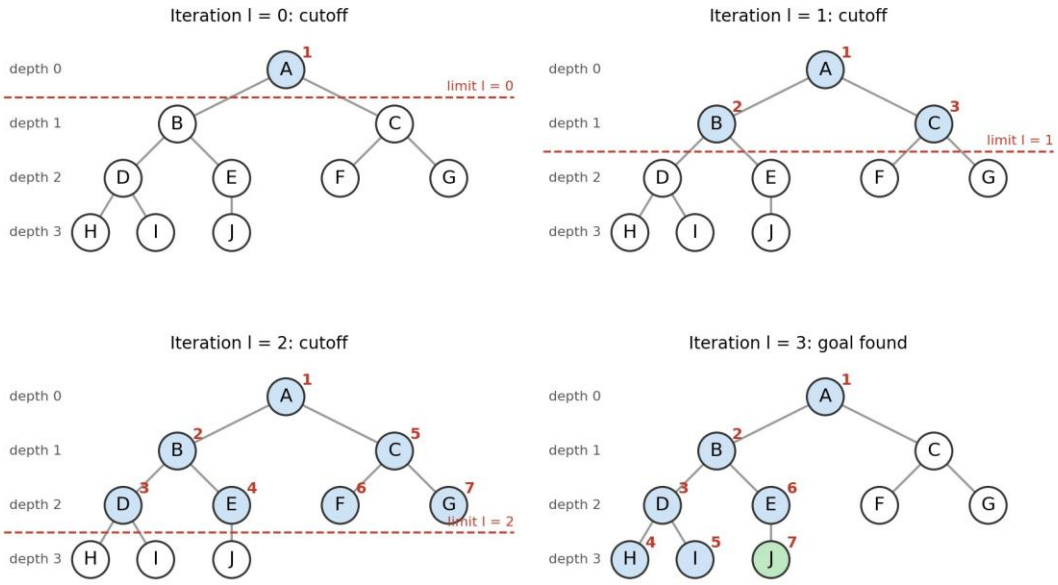
Idea. IDDFS removes the need to guess the limit. It runs DLS with l = 0, then l = 1, then l = 2 and so on, increasing the limit by one each time, until a solution turns up. In this way it takes the low memory use of DFS and the 'shallowest goal first' behaviour of BFS.

```
 function IDDFS(start, goal):
 for limit = 0, 1, 2, 3, ... :
         result <- DLS(start, goal, limit)
if result != cutoff:                return
result
```

Trace on the example.

Limit	Nodes visited (in order)	Result
0	A	cutoff
1	A - B - C	cutoff
2	A - B - D - E - C - F - G	cutoff
3	A - B - D - H - I - E - J	goal found

Figure 5: Iterative deepening - the same tree searched four times



Is repeating work really that wasteful? At first glance, searching the upper levels again and again looks silly. In the example IDDFS visited 18 nodes in total, although the tree only has 10. But in a tree, most nodes sit at the bottom level, so the repeated top levels cost very little. A standard 2 d calculation makes this clear. For b = 10 and d = 5, the formula N = d*b + (d-1)*b + ... + 1*b gives 2 d 123,450 nodes for IDDFS, while BFS generates b + b + ... + b = 111,110 nodes. That is an overhead of only about 11% (my own arithmetic from these formulas; textbooks give the same figures).

Properties.

- **Complete:** yes, when the branching factor is finite.
- **Optimal:** yes when all step costs are equal, because it always finds the shallowest goal.
- **Time:** $O(b^d)$.
- **Space:** O(b*d).

Where it is used. It is the usual choice for uninformed search when the search space is large and the depth of the answer is unknown. Game-playing programs also use the same idea: they search one level deeper each time until the time for the move runs out.

6. Uniform-Cost Search (UCS)

Idea. BFS counts steps, but in real problems steps are rarely equal. A motorway kilometre and a mountain-road kilometre do not cost the same time or fuel. UCS fixes this by always expanding the frontier node with the **lowest path cost so far**, written g(n). It spreads out in 'cost contours' instead of depth levels.

Data structure. A priority queue ordered by g(n). If a cheaper path to a node already on the frontier is found, the old cost is replaced. Two details matter. The goal test is done when a node is *expanded*, not when it is generated, because the first path found to the goal might not be the cheapest one. And the replacement step is what lets UCS fix an early expensive path.

```
function UCS(start, goal):     frontier <- priority queue ordered by
g,
```

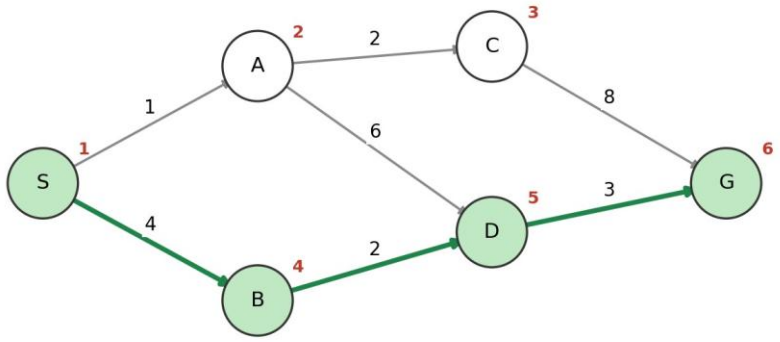
Step	Expanded	g(n)	Frontier afterwards, as node(cost)
1	S	0	A(1), B(4)
2	A	1	C(3), B(4), D(7)
3	C	3	B(4), D(7), G(11)
4	B	4	D(6), G(11)
5	D	6	G(9)
6	G	9	goal reached

```
containing start (g = 0)     explored <- empty set     while frontier
is not empty:
        node <- frontier.pop_lowest_g()         if node
== goal: return path to node         add node to explored
for each (child, step_cost) of node:             new_g <-
g(node) + step_cost             if child not in explored
and not in frontier:
                frontier.insert(child, new_g)
else if child in frontier with higher g:
                replace it with new_g
return failure
```

Example. The graph in Figure 6 has start S and goal G. Edge labels are step costs.

Look at steps 4 and 5. D was first

Figure 6: Weighted graph for UCS (red = order of expansion, green = cheapest path)



reached through A with cost 7, but after B was expanded a cheaper route S - B - D with cost 6 was found, so the entry for D was updated. G was first reached with cost 11 and later improved to 9. UCS only stops when G is actually removed from the queue, so the answer is the path **S - B - D - G** with total cost **9**. By comparison, BFS on the same graph returns S - A - C - G with cost 11, since it only looks at the number of edges and all the routes here happen to have three.

Properties.

- **Complete:** yes, provided every step cost is at least some small positive number epsilon. Zero-cost loops can trap it.
- **Optimal:** yes. Nodes are expanded in order of increasing path cost, so the first goal removed from the queue is a cheapest one.
- **Time and space:** $O(b^{1 + \text{floor}(C^*/\text{epsilon})})$. This can be much worse than b^d when there are many cheap steps.

Where it is used. Route planning with distances or travel times, network routing and any problem where actions have different costs. UCS is very close to Dijkstra's shortest-path algorithm, the main difference being that UCS stops as soon as the goal is reached.

7. Comparison of the Five Algorithms

The table below collects the results. Entries follow the standard textbook analysis (see the reference at the end).

Algorithm	Complete?	Optimal?	Time	Space
BFS	Yes (b finite)	Yes, if step costs are equal	$O(b^d)$	$O(b^d)$
DFS	No (loops / infinite depth)	No	$O(b^m)$	O(b*m)
DLS	No (if l < d)	No	$O(b^l)$	O(b*l)
IDDFS	Yes (b finite)	Yes, if step costs are equal	$O(b^d)$	O(b*d)
UCS	Yes (step cost >= epsilon)	Yes	$O(b^{1+C^*/e})$	$O(b^{1+C^*/e})$

For UCS, the exponent is really 1 + floor(C*/epsilon), where e stands for epsilon. b = branching factor, d = goal depth, m = maximum depth, l = depth limit.

Figure 7: Growth of the memory formulas, b = 3

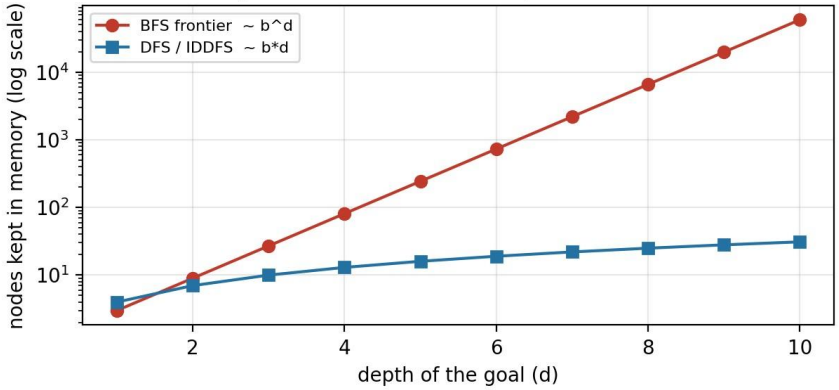


Figure 7 plots only the two memory formulas, it is not a measurement of a real program. Even for a small branching factor of 3, the BFS formula explodes (note the log scale) while the DFS-style formula grows slowly.

8. Which One Should I Choose?

- Step costs are equal, the state space is small and the shortest path matters: **BFS**.
- Memory is limited and any solution will do: **DFS**.
- A sensible maximum depth is known from the problem: **DLS**.
- The space is big, the goal depth is unknown and memory is tight: **IDDFS**.
- Actions have different costs and the cheapest answer is needed: **UCS**.

The weakness shared by all five is that they are blind. They treat a node that is one step from the goal exactly like a node that is miles away. On large problems this makes them slow, which is the motivation for informed methods such as A*.

9. Conclusion

Uninformed search shows that a complete and quite effective problem solver can be built from nothing but a problem definition and a rule for choosing the next node. BFS guarantees the shallowest solution but needs a lot of memory. DFS needs very little memory but may wander or return a poor answer. DLS adds a limit to make DFS safe, and IDDFS removes the need to guess that limit while keeping memory low. UCS extends the BFS idea to problems with unequal costs and gives the cheapest path. No single method wins everywhere, so the choice depends on the cost structure, the memory available and how much is known about the depth of the goal.